



TBA024

TEORIA DE ALGORITMOS

Trabajo Práctico N°1

- Curso Echevarría 2do Cuatrimestre 2026 -

Alumno	Padrón
Alzogaray, Joaquín	108905
González Hidalgo, Eduardo Eliezer	110006
Nelson Avila	100244
Ignacio Oviedo	109821

Fecha:
04/10/26

Índice

1. Introducción al Problema de la Mochila	3
1.1. Formulación Matemática	3
2. Fuerza bruta vs. Backtracking	4
2.1. Supuestos	4
2.2. Diseño	5
2.2.1. Pseudocódigo: Fuerza bruta	5
2.2.2. Pseudocódigo: Backtracking	5
2.2.3. Estructuras de datos utilizadas	7
2.3. Seguimiento	7
2.3.1. Fuerza bruta	8
2.3.2. Backtracking	8
2.4. Complejidad	9
2.4.1. Fuerza bruta	9
2.4.2. Backtracking	10
2.5. Sets de datos	10
2.6. Tiempos de ejecución	11
2.7. Informe de resultados	13
3. Algoritmo Greedy	16
3.1. Supuestos	16
3.2. Diseño	16
3.2.1. Estructuras de Datos Utilizadas	16
3.2.2. Pseudocódigo	17
3.2.3. Garantía de Calidad (1/2) y Demostración Matemática	18
3.3. Seguimiento con Set Reducido (Activación de la Garantía de Calidad)	18
3.4. Complejidad Temporal	20
3.5. Sets de Datos	21
3.6. Tiempos de Ejecución	22
3.7. Informe de Resultados	23
4. Programación Dinámica	25
4.1. Identificación de supuestos, condiciones, limitaciones y/o premisas. . .	25
4.1.1. Consideraciones del algoritmo tradicional.	26
4.1.2. Consideraciones del algoritmo alternativo.	26
4.2. Diseño.	27
4.2.1. Pseudocódigo del algoritmo tradicional.	27
4.2.2. Pseudocódigo del algoritmo alternativo.	28
4.2.3. Estructuras de datos utilizadas.	29

4.3.	Ejemplos de seguimiento.	30
4.3.1.	Ejemplo para el algoritmo tradicional.	30
4.3.2.	Ejemplo para el algoritmo alternativo.	34
4.4.	Complejidad temporal.	37
4.4.1.	Complejidad del algoritmo tradicional.	38
4.4.2.	Complejidad del algoritmo alternativo.	39
4.5.	Tiempos de ejecución.	40
4.5.1.	Set de Datos.	40
4.5.2.	Resultados.	42
4.5.3.	Gráficos.	43
4.6.	Informe de resultados comparativos: tiempos medidos vs tiempos teóricos.	44
5.	Programación Lineal Entera	47
5.1.	Supuestos	47
5.2.	Diseño	47
5.2.1.	Modelado Matemático	47
5.2.2.	Pseudocódigo	48
5.3.	Seguimiento con Set Reducido	48
5.4.	Complejidad Temporal	50
5.5.	Tiempos de Ejecución	51
5.6.	Informe de Resultados	53
6.	Conclusión General	54
6.1.	Comparativa de Complejidades Teóricas	54

1. Introducción al Problema de la Mochila

El **Problema de la Mochila 0/1** (*0/1 Knapsack Problem*) es un problema canónico de optimización combinatoria.

1.1. Formulación Matemática

Se cuenta con:

- Una capacidad máxima de carga $W \in \mathbb{Z}^+$.
- Un conjunto de n objetos disponibles $S = \{1, 2, \dots, n\}$.
- Cada objeto $i \in S$ tiene asignado un peso $w_i \in \mathbb{Z}^+$ y un beneficio o valor $v_i \in \mathbb{Z}^+$.

El objetivo es determinar la asignación binaria $x_i \in \{0, 1\}$ para cada objeto i , maximizando el valor total obtenido sin violar la restricción de capacidad:

$$\max \sum_{i=1}^n v_i x_i \quad \text{sueto a} \quad \sum_{i=1}^n w_i x_i \leq W, \quad x_i \in \{0, 1\}$$

Una vez presentado el problema, se mostrará cómo se resuelve el problema de la mochila con distintos algorítmicos, con el objetivo de comparar el tiempo de ejecución y la calidad de la solución que ofrece cada uno.

Para cada algoritmo se presentan los supuestos, el diseño (pseudocódigo y estructuras de datos), un seguimiento sobre un conjunto reducido de datos, el análisis de complejidad temporal, los tiempos de ejecución medidos y su comparación con la curva teórica.

2. Fuerza bruta vs. Backtracking

Problema de la mochila 0/1: dado un conjunto de n elementos, cada uno con un peso y un valor, y una mochila de capacidad W , elegir un subconjunto de elementos cuyo peso total no supere W y cuyo valor total sea máximo.

Se implementaron dos algoritmos en Python (`codigo.py`):

- **fuerzaBruta**: recorre todos los subconjuntos factibles y se queda con el de mayor valor.
- **backtracking**: recorre el mismo árbol de búsqueda, pero poda las ramas que no pueden superar a la mejor solución encontrada hasta el momento.

2.1. Supuestos

Identificación de supuestos, condiciones, limitaciones y/o premisas bajo los cuales funcionará el algoritmo desarrollado.

- **Mochila 0/1**: cada elemento se toma entero o no se toma. No se permiten fracciones ni elementos repetidos.
- **Datos enteros y positivos**: la capacidad de la mochila, los pesos y los valores son números enteros mayores que cero. Con valores negativos o nulos, la cota superior del backtracking dejaría de ser válida.
- **Formato de entrada**: los datos se leen de un archivo de texto con el formato generado por `crear_mochila.py`: la primera línea es la capacidad y cada línea siguiente es un par `peso,valor`.
- **Elementos independientes**: el valor total es la suma de los valores de los elementos elegidos y no hay restricciones entre elementos (incompatibilidades, dependencias, etc.).
- **Un solo óptimo informado**: si hay varias soluciones con el mismo valor máximo, se devuelve la primera encontrada según el orden de los elementos en el archivo.
- **Tamaño de entrada limitado**: los dos algoritmos son exponenciales en el peor caso, así que solo son prácticos para n chico. En la máquina de prueba, la fuerza bruta tarda ~ 76 s con $n = 28$. El backtracking llega a $n = 40$ en ~ 17 s, pero su tiempo depende mucho de los datos: con $n = 42$ una instancia superó los 9 minutos.
- **Profundidad de recursión**: la profundidad máxima es $n+1$, muy por debajo del límite de recursión por defecto de Python (1000) para los tamaños usados.

- **Orden de los elementos:** ninguno de los dos algoritmos ordena los elementos. El backtracking poda más o menos según el orden en que aparecen, pero el resultado (el valor óptimo) no cambia.

2.2. Diseño

2.2.1. Pseudocódigo: Fuerza bruta

```
funcion fuerzaBruta(mochila, elementos, mejor):
    si mochila.valor > mejor.valor:
        mejor <- copia(mochila)

    pendientes <- copia(elementos)
    para cada elemento en elementos:
        pivote <- quitarPrimero(pendientes)    // los que quedan son los
                                                // posteriores al pivote
        si pivote.peso <= mochila.pesoDisponible:
            agregar(mochila, pivote)
            resultado <- fuerzaBruta(copia(mochila), copia(pendientes), mejor)
            si resultado.valor > mejor.valor:
                mejor <- resultado
            quitarUltimo(mochila)                // deshace la eleccion
    devolver mejor

// llamada inicial
mejor <- fuerzaBruta(Mochila(W), elementos, Mochila(W))
```

2.2.2. Pseudocódigo: Backtracking

```
funcion cotaSuperior(mochila, elementos):
    cota <- mochila.valor
    para cada elemento en elementos:
        si elemento.peso <= mochila.pesoDisponible:
            cota <- cota + elemento.valor
    devolver cota

funcion backtracking(mochila, elementos, mejor):
    si mochila.valor > mejor.valor:
```

```
mejor <- copia(mochila)

si cotaSuperior(mochila, elementos) <= mejor.valor: // poda de la rama
  devolver mejor

pendientes <- copia(elementos)
para cada elemento en elementos:
  pivote <- quitarPrimero(pendientes)
  si pivote.peso <= mochila.pesoDisponible:
    agregar(mochila, pivote)
    resultado <- backtracking(mochila, pendientes, mejor)
    si resultado.valor > mejor.valor:
      mejor <- resultado
    quitarUltimo(mochila)
    si cotaSuperior(mochila, pendientes) <= mejor.valor:
      cortar el ciclo // poda del resto de los hermanos
devolver mejor
```

La cota superior es la cantidad más optimista que se puede alcanzar desde el estado actual: el valor que ya hay en la mochila más el valor de cada elemento restante que entraría por sí solo en el espacio libre. Ningún subconjunto factible de los elementos restantes puede superarla, así que si no supera al mejor valor ya encontrado, la rama puede descartarse sin perder el óptimo.

La segunda poda (después de deshacer la elección del pivote) vale para todos los hermanos que quedan en el ciclo, porque todos son subconjuntos de **pendientes**.

2.2.3. Estructuras de datos utilizadas

Estructura	Implementación	Uso
Elemento	Clase con dos enteros: peso y valor .	Representa cada elemento del archivo.
Mochila	Clase con peso (capacidad), pesoDisponible , valorActual e itemsDentro (lista de Python).	Estado parcial de la solución. itemsDentro funciona como pila : agregarElemento hace append y quitarElemento hace pop , ambos $O(1)$. pesoDisponible y valorActual se actualizan en cada operación para no recalcularlos. copiar() cuesta $O(n)$.
Lista de elementos	list de Python con objetos Elemento .	Elementos a considerar. Se copia (list(...)) y se consume con pop(0) para que cada pivote solo combine con los elementos posteriores, y así cada subconjunto se genere una sola vez.
Mejor solución	Una Mochila copiada.	Guarda la mejor solución encontrada y se pasa por parámetro en la recursión.
Pila de llamadas	Recursión.	Representa el camino actual en el árbol de decisión. Profundidad máxima $n + 1$.

La diferencia entre los dos algoritmos en el manejo de memoria: la fuerza bruta pasa una **copia** de la mochila y de la lista a cada llamada recursiva, mientras que el backtracking trabaja sobre **la misma mochila**: agrega, llama y deshace. Sólo la copia cuando encuentra una mejor solución.

2.3. Seguimiento

Ejemplo de seguimiento con un set de datos reducido. Se usa **Mochilas/mochila5.txt**, con capacidad $W = 250$ y 5 elementos:

Elemento	Peso	Valor
A	74	140
B	171	65
C	127	470
D	27	303
E	196	410

2.3.1. Fuerza bruta

Cada línea es una llamada recursiva (un nodo del árbol). La sangría indica la profundidad.

```
{ }      peso=0   valor=0   restantes=[A,B,C,D,E]
{A}      peso=74  valor=140 restantes=[B,C,D,E]  -> NUEVO MEJOR (140)
  {A,B}  peso=245 valor=205 restantes=[C,D,E]    -> NUEVO MEJOR (205)
        C, D, E no entran (disponible = 5)
  {A,C}  peso=201 valor=610 restantes=[D,E]      -> NUEVO MEJOR (610)
        {A,C,D} peso=228 valor=913 restantes=[E] -> NUEVO MEJOR (913)
        E no entra (disponible = 22)
        E no entra (disponible = 49)
  {A,D}  peso=101 valor=443 restantes=[E]
        E no entra (disponible = 149)
        E no entra (disponible = 176)
{B}      peso=171 valor=65  restantes=[C,D,E]
        C no entra (disponible = 79)
  {B,D}  peso=198 valor=368 restantes=[E]
        E no entra (disponible = 52)
        E no entra (disponible = 79)
{C}      peso=127 valor=470 restantes=[D,E]
  {C,D}  peso=154 valor=773 restantes=[E]
        E no entra (disponible = 96)
        E no entra (disponible = 123)
{D}      peso=27   valor=303 restantes=[E]
  {D,E}  peso=223 valor=713 restantes=[]
{E}      peso=196 valor=410 restantes=[]
```

Recorre los **13 subconjuntos factibles** (de los $2^5 = 32$ posibles) y devuelve $\{A, C, D\}$, con peso 228 y valor **913**.

2.3.2. Backtracking

Se agrega la cota superior de cada nodo y las podas.

```

{}          valor=0   restantes=[A,B,C,D,E]   cota=1388
{A}          valor=140 restantes=[B,C,D,E]     cota=978  -> NUEVO MEJOR (140)
{A,B}        valor=205 restantes=[C,D,E]       cota=205  -> NUEVO MEJOR (205)
                                PODA: cota 205 <= mejor 205
{A,C}        valor=610 restantes=[D,E]         cota=913  -> NUEVO MEJOR (610)
{A,C,D}      valor=913 restantes=[E]           cota=913  -> NUEVO MEJOR (913)
                                PODA: cota 913 <= mejor 913
    corte del ciclo: sin D, cota con [E] = 610 <= 913   (no visita {A,C,E})
    corte del ciclo: sin C, cota con [D,E] = 443 <= 913 (no visita {A,D})
{B}          valor=65  restantes=[C,D,E]       cota=368  PODA: 368 <= 913
{C}          valor=470 restantes=[D,E]         cota=773  PODA: 773 <= 913
    corte del ciclo: sin C, cota con [D,E] = 713 <= 913 (no visita {D}, {E})

```

Llega al mismo resultado, $\{A, C, D\}$ con valor **913**, visitando **7 nodos en vez de 13**. Una vez encontrado el óptimo (913), las cotas de las ramas restantes ($\{B\}$: 368, $\{C\}$: 773, $\{D, E\}$: 713) no lo pueden superar y se descartan sin explorarlas.

2.4. Complejidad

Análisis de la complejidad temporal a partir del pseudocódigo.

2.4.1. Fuerza bruta

Cantidad de nodos. Cada llamada recursiva corresponde a un subconjunto factible distinto, porque el pivote solo se combina con los elementos que vienen después de él. Así, cada subconjunto se genera una sola vez, en orden creciente de índices. Hay a lo sumo 2^n subconjuntos. El peor caso es cuando todos los elementos entran juntos ($W \geq \sum \text{pesos}$) y el árbol tiene exactamente 2^n nodos.

Costo por nodo.

- Comparar con el mejor: $O(1)$. Copiar la mochila cuando mejora: $O(n)$.
- Por cada hijo que se crea: `agregarElemento` y `quitarElemento` en $O(1)$, `copia(mochila)` en $O(n)$ y `copia(pendientes)` en $O(n)$. Ese costo se le asigna al hijo, así que cada nodo carga con $O(n)$.
- El ciclo con `quitarPrimero` (`pop(0)`) cuesta $O(k^2)$ en un nodo con k elementos restantes. Pero hay 2^{n-1-k} nodos con k restantes, y

$$\sum_k 2^{n-1-k} \cdot k^2 = 2^{n-1} \sum_k \frac{k^2}{2^k} = O(2^n).$$

Ese término queda absorbido.

Recurrencia. Llamando $T(k)$ al costo de resolver con k elementos restantes, con n fijo:

$$T(0) = c \cdot n$$

$$T(k) = 2T(k-1) + c \cdot n \quad (\text{incluir o no el primer elemento disponible, más } O(n) \text{ de copias})$$

$$\Rightarrow T(n) = c \cdot n \cdot (2^{n+1} - 1)$$

$$T_{FB}(n) = O(n \cdot 2^n)$$

Memoria: $O(n^2)$, porque cada uno de los hasta n niveles de la recursión guarda una copia de la mochila y de la lista.

2.4.2. Backtracking

El árbol de búsqueda es el mismo. Los cambios son:

- Ya no hay copias por hijo: agregar y deshacer cuestan $O(1)$. La mochila solo se copia cuando mejora el óptimo, $O(n)$.
- Se agrega `cotaSuperior`, que recorre los elementos restantes: $O(n)$ al entrar al nodo y $O(n)$ después de cada hijo, costo que se le asigna al hijo.

Entonces cada nodo sigue costando $O(n)$. **En el peor caso la poda no descarta ninguna rama** (por ejemplo, si cada nueva solución mejora apenas la anterior y la cota nunca baja del óptimo actual), se visitan los 2^n nodos y queda la misma recurrencia:

$$T_{BT}(n) = O(n \cdot 2^n)$$

La poda **no mejora el orden del peor caso**, pero en la práctica reduce significativamente la cantidad de nodos visitados. En el seguimiento pasó de 13 a 7 nodos, y en las mediciones de la sección 6 se ve que el ahorro crece con n . **Memoria:** $O(n)$, por la profundidad de recursión más una copia de la mejor solución.

2.5. Sets de datos

Se adjuntan los sets de datos utilizados y se indican los criterios con que se tomaron.

Generación. Los sets los genera `grafico.py` (función `crearElementos`) con la misma distribución que `crear_mochila.py`:

- Peso de cada elemento: entero uniforme entre 1 y 200.
- Valor de cada elemento: entero uniforme entre 1 y 1000.
- Capacidad de la mochila: $W = 50 \cdot n$. Como el peso medio es ~ 100 , entra aproximadamente la mitad de los elementos.

Cada set se genera con una **semilla fija**, `random.Random(semilla)`, así que cualquier corrida los reproduce exactamente.

Tamaños elegidos. Se usan tamaños pares, buscando que se vea el crecimiento exponencial sin que la corrida total supere unos 10 minutos:

Algoritmo	Tamaños (n)	Motivo del límite
Fuerza bruta	2, 4, ..., 28	Con $n = 28$ tarda ~ 76 s por corrida. Con $n = 30$ se estiman ~ 5 min por corrida.
Backtracking	2, 4, ..., 40	Con $n = 42$ una instancia superó los 9 minutos sin terminar.

Para $n \leq 28$ los dos algoritmos usan **exactamente los mismos sets**, así la comparación es directa.

Semillas.

- Por defecto, la semilla de cada tamaño es $42 + n$.
- Para $n \geq 30$ (solo backtracking), el tiempo variaba mucho entre instancias del mismo tamaño: por ejemplo, con $n = 34$ fue de 0,27 s a 56 s. Para que cada tamaño quede representado por una instancia típica, se generaron **7 instancias por tamaño** (semillas $42 + n + 1000 \cdot k$, con $k = 0 \dots 6$) y se eligió **la de tiempo mediano**. Las semillas elegidas quedaron en `SEMILLAS_ELEGIDAS` de `grafico.py`: $n = 32 \rightarrow 3074$, $n = 34 \rightarrow 1076$, $n = 36 \rightarrow 3078$, $n = 38 \rightarrow 5080$. Para $n = 30$ y $n = 40$, la mediana coincidió con la semilla por defecto.

Seguimiento. Para el ejemplo de la sección 3 se usó `Mochilas/mochila5.txt`, generado con `crear_mochila.py`.

2.6. Tiempos de ejecución

Medición de los tiempos de ejecución de cada set de datos.

Metodología.

- **Medición:** `time.perf_counter()` alrededor de la llamada a cada algoritmo.

- **Repeticiones:** cada set se ejecuta 3 veces y se toma el **mínimo**, para descartar interrupciones del sistema operativo.
- **Entorno:** Intel Core i5-14600KF, Windows 11, Python 3.14.6.
- **Curva teórica:** cantidad de operaciones del orden calculado ($n \cdot 2^n$) multiplicada por un tiempo asumido por operación elemental de **10 ns**: $t_{\text{teórico}}(n) = n \cdot 2^n \cdot 10^{-8}$ s. Se usa la misma curva para los dos algoritmos, porque los dos tienen el mismo orden en el peor caso.

Los resultados completos están en `tiempos.csv`.

n	Teórico	Fuerza bruta (s)	Backtracking (s)	FB / BT	BT / teórico
2	0,00000008	0,00000007	0,00000006	1	—
4	0,00000006	0,00000075	0,00000045	2	—
6	0,00000038	0,0000136	0,00000075	2	—
8	0,0000205	0,0000648	0,0000165	4	80,6 %
10	0,000102	0,000335	0,0000466	7	45,5 %
12	0,000492	0,000969	0,000124	8	25,3 %
14	0,002294	0,003701	0,000214	17	9,35 %
16	0,010486	0,018880	0,000399	47	3,80 %
18	0,047186	0,081652	0,000922	89	1,95 %
20	0,209715	0,421127	0,002179	193	1,04 %
22	0,922747	1,198938	0,003618	331	0,39 %
24	4,026532	4,248935	0,003272	1299	0,081 %
26	17,448305	7,237426	0,131278	55	0,75 %
28	75,161928	76,358396	0,805558	95	1,07 %
30	322,12	—	0,699305	—	0,22 %
32	1 374,39	—	0,289191	—	0,021 %
34	5 841,16	—	3,703661	—	0,063 %
36	24 739,01	—	7,211016	—	0,029 %
38	104 453,60	—	21,866105	—	0,021 %
40	439 804,65	—	17,087767	—	0,0039 %

Para $n \leq 6$ la columna BT / teórico no se informa: los tiempos son de microsegundos, dominados por el costo fijo de las llamadas de Python, y el cociente no es representativo.

- **Izquierda:** fuerza bruta, tiempo medido contra la curva teórica $O(n \cdot 2^n)$.
- **Centro:** backtracking, tiempo medido contra la misma curva teórica. El eje Y se limita al rango de los tiempos medidos para que se vea su evolución; la curva teórica sale del gráfico alrededor de $n = 26$.

- **Derecha:** comparación de los tiempos medidos de los dos algoritmos, en la misma escala.

2.7. Informe de resultados

Comparación de los tiempos de ejecución y análisis de si se corresponden con la complejidad determinada inicialmente. El gráfico comparativo incluye tanto la curva con los valores medidos como la curva teórica de ajuste.

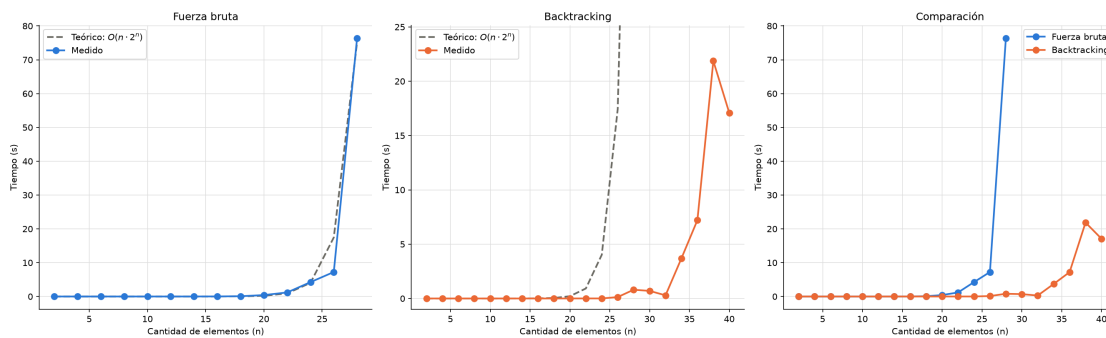


Figura 1: Tiempos de ejecución medidos experimentalmente (puntos azules y naranjas) contrastados con la curva teórica (línea punteada/discontinua de ajuste por mínimos cuadrados)

Correctitud. Los dos algoritmos devuelven el mismo valor óptimo en todos los sets en que se pudo comparar ($n = 2$ a 22 se verificó explícitamente, además del seguimiento con `mochila5.txt`). El backtracking llega al mismo resultado que la fuerza bruta en menos tiempo, como pide el enunciado.

Fuerza bruta: sí se corresponde con $O(n \cdot 2^n)$.

- La curva medida sigue a la teórica muy de cerca: 4,25 s contra 4,03 s en $n = 24$, y 76,4 s contra 75,2 s en $n = 28$. Esto confirma, además, que 10 ns por operación es una buena estimación para esta máquina.
- Cada dos elementos más, el tiempo se multiplica por un factor cercano a 4, como predice 2^n . Por ejemplo, de $n = 16$ a $n = 18$ pasa de 0,019 s a 0,082 s ($\times 4,3$) y de $n = 18$ a $n = 20$, de 0,082 s a 0,42 s ($\times 5,2$).
- La desviación más visible es $n = 26$ (7,2 s, por debajo de los 17,4 s teóricos). La fuerza bruta no recorre los 2^n subconjuntos, sino solo los **factibles**, y su cantidad depende de los pesos concretos de cada set. La curva teórica corres-

ponde al peor caso (todos los elementos entran), así que funciona como cota superior del comportamiento real, con oscilaciones según los datos.

Backtracking: cumple la cota $O(n \cdot 2^n)$, pero en la práctica queda muy por debajo.

- El tiempo medido es una fracción cada vez menor de la curva teórica: $\sim 45\%$ en $n = 10$, $\sim 1\%$ en $n = 20$ y $0,004\%$ en $n = 40$. Para $n = 40$ el modelo teórico predice unos 5 días (439 805 s) y el backtracking terminó en 17 s.
- Esto no contradice el análisis de complejidad. $O(n \cdot 2^n)$ es el **peor caso**, en el que la poda no descarta nada, y con datos aleatorios como los de `crear_mochila.py` la poda elimina la gran mayoría de las ramas. Cuanto más grande es el árbol, más nodos quedan debajo de cada rama podada, y por eso la proporción de trabajo ahorrado crece con n .
- El crecimiento medido sigue siendo exponencial, pero con una base efectiva mucho menor que 2. Entre $n = 20$ y $n = 40$ el tiempo crece unas 8 000 veces (de 0,0022 s a 17 s), mientras que $n \cdot 2^n$ crece unas 2 millones de veces.
- La curva no es estrictamente creciente: $n = 32$ (0,29 s) tarda menos que $n = 30$ (0,70 s), y $n = 40$ (17,1 s) menos que $n = 38$ (21,9 s). La cantidad de podas depende de los datos de cada instancia: cuándo aparece una buena solución, qué tan ajustada es la cota y el orden de los elementos. Incluso eligiendo la instancia mediana entre 7, la variabilidad entre instancias del mismo tamaño es muy alta (en $n = 34$, de 0,27 s a 56 s). El tiempo del backtracking depende más de los datos que del tamaño.

Comparación entre los dos algoritmos.

- Para $n \leq 6$ los dos tardan lo mismo en la práctica (microsegundos). El árbol es tan chico que la poda casi no tiene qué descartar, y cada nodo del backtracking hace más trabajo, porque calcula la cota.
- A partir de $n \approx 10$ la diferencia empieza a crecer rápido: el backtracking es ~ 17 veces más rápido en $n = 14$, ~ 190 en $n = 20$ y ~ 95 en $n = 28$, con picos de ~ 1300 ($n = 24$) según la instancia.
- En la práctica, esto amplía el tamaño de problema que se puede resolver en un tiempo razonable: con unos 20 s por instancia, la fuerza bruta llega hasta $n \approx 26$ y el backtracking hasta $n \approx 38$.

Conclusión. Los dos algoritmos tienen complejidad temporal $O(n \cdot 2^n)$ en el peor caso, y los tiempos medidos de la fuerza bruta se ajustan muy bien a esa curva. El backtracking respeta la misma cota, pero la poda por cota superior hace que su comportamiento real sea órdenes de magnitud mejor. A cambio, su tiempo es menos predecible: depende de los datos de cada instancia y no solo de n .

3. Algoritmo Greedy

3.1. Supuestos

Para el diseño, implementación y análisis del algoritmo voraz se establecen los siguientes supuestos, condiciones y limitaciones:

1. **Indivisibilidad de objetos (0/1):** Cada elemento debe incluirse íntegramente o descartarse ($x_i \in \{0, 1\}$). No se permite fraccionamiento.
2. **Positividad de parámetros:** Tanto la capacidad de la mochila W como los pesos w_i y los beneficios v_i de los n objetos son cantidades enteras estrictamente positivas ($W > 0$, $w_i > 0$, $v_i > 0$).
3. **Objetos viables:** Todo elemento cuyo peso individual sea estrictamente mayor que la capacidad total de la mochila ($w_i > W$) no puede formar parte de ninguna solución factible y es descartado en la fase de preprocesamiento.
4. **Heurística voraz y necesidad de garantía:** Se asume que una heurística voraz estándar basada puramente en la densidad de valor (v_i/w_i) puede resultar arbitrariamente mala en la mochila 0/1. Por ello, el algoritmo incorpora un mecanismo de seguridad con **garantía de aproximación de factor 2** (garantía de calidad $1/2$), asegurando formalmente que la ganancia obtenida nunca sea inferior a la mitad del óptimo global:

$$\text{Solución} \geq \frac{1}{2} \cdot \text{OPT}$$

3.2. Diseño

3.2.1. Estructuras de Datos Utilizadas

- **Elemento:** Estructura de datos que almacena el identificador único `id`, el **peso** (w_i), el **valor** (v_i) y calcula dinámicamente la propiedad `ratio = valor / peso` (densidad de beneficio por unidad de peso).
- **ResultadoMochila:** Estructura que encapsula el resultado devuelto por la función:
 - `valor_total`: suma de los beneficios obtenidos.
 - `peso_total`: peso acumulado consumido de la mochila.
 - `elementos_seleccionados`: lista con las referencias a los objetos incluidos.

- **actuo_garantia:** variable booleana que indica si la solución final proviene del elemento crítico o del conjunto voraz tradicional.
- **Lista dinámica:** Arreglo contiguo en memoria para almacenar la colección de elementos y permitir el ordenamiento eficiente en $\mathcal{O}(n \log n)$.

3.2.2. Pseudocódigo

```
1  Algoritmo MochilaGreedyConGarantia(W, Elementos):
2      Candidatos <- [e en Elementos tal que e.peso ≤ W]
3      Si Candidatos es vacio:
4          Retornar ResultadoMochila(0, 0, [], Falso)
5
6      Ordenar Candidatos descendientemente segun (e.valor
7          / e.peso)
8
9      SolucionVoraz <- []
10     PesoVoraz <- 0
11     ValorVoraz <- 0
12     ElementoCritico <- Nulo
13
14     Para cada e en Candidatos:
15         Si PesoVoraz + e.peso ≤ W:
16             SolucionVoraz.Agregar(e)
17             PesoVoraz <- PesoVoraz + e.peso
18             ValorVoraz <- ValorVoraz + e.valor
19         Sino si ElementoCritico es Nulo:
20             ElementoCritico <- e
21
22     Si ElementoCritico ≠ Nulo y ElementoCritico.valor >
23         ValorVoraz:
24         Retornar ResultadoMochila(ElementoCritico.valor,
25             ElementoCritico.peso,
26             [ElementoCritico],
27             Verdadero)
28
29     Retornar ResultadoMochila(ValorVoraz,
30         PesoVoraz,
31         SolucionVoraz,
32         Falso)
```

3.2.3. Garantía de Calidad (1/2) y Demostración Matemática

Para justificar que el algoritmo satisface la cota $\text{Solución} \geq \frac{1}{2}OPT$:

1. Sea OPT el valor de la solución óptima del problema de la mochila binaria 0/1.
2. Consideremos la **mochila fraccionaria** (relajación continua donde $x_i \in [0, 1]$), cuyo valor óptimo denotamos OPT_{frac} . Como toda solución binaria es una solución fraccionaria factible:

$$OPT \leq OPT_{\text{frac}}$$

3. En la versión fraccionaria, el algoritmo voraz es exacto: incorpora todos los elementos del conjunto voraz S_1 (que aportan valor P_1) más una fracción continua $\alpha \in [0, 1)$ del elemento crítico c :

$$OPT_{\text{frac}} = P_1 + \alpha \cdot v_c < P_1 + v_c$$

4. Por transitividad:

$$OPT < P_1 + v_c$$

5. El algoritmo elige el máximo entre ambos valores: $\text{Solución} = \max(P_1, v_c)$. Por propiedad del promedio entre dos números reales:

$$\max(P_1, v_c) \geq \frac{P_1 + v_c}{2}$$

6. Aplicando la cota de OPT :

$$\text{Solución} \geq \frac{P_1 + v_c}{2} > \frac{OPT}{2}$$

Esto demuestra que **el beneficio devuelto por el algoritmo es estrictamente superior al 50 % del óptimo**, logrando la garantía exigida.

3.3. Seguimiento con Set Reducido (Activación de la Garantía de Calidad)

Para verificar el algoritmo y demostrar la activación de la garantía (punto 4 del enunciado), se aplica el seguimiento paso a paso sobre un set manual de 7 elementos con capacidad $W = 100$:

Datos de Entrada

ID	Peso (w_i)	Beneficio (v_i)	Ratio (v_i/w_i)	Estado Inicial
1	5	10	2.00	Candidato
2	5	10	2.00	Candidato
3	10	18	1.80	Candidato
4	10	16	1.60	Candidato
5	75	110	1.47	Candidato
6	40	40	1.00	Candidato
7	40	36	0.90	Candidato

Traza de Ejecución Paso a Paso

1. Llenado voraz:

- **Elemento 1:** $w_1 = 5 \leq 100$. Entra. $\text{PesoVoraz} = 5$, $\text{ValorVoraz} = 10$. Capacidad remanente: 95.
- **Elemento 2:** $w_2 = 5 \leq 95$. Entra. $\text{PesoVoraz} = 10$, $\text{ValorVoraz} = 20$. Capacidad remanente: 90.
- **Elemento 3:** $w_3 = 10 \leq 90$. Entra. $\text{PesoVoraz} = 20$, $\text{ValorVoraz} = 38$. Capacidad remanente: 80.
- **Elemento 4:** $w_4 = 10 \leq 80$. Entra. $\text{PesoVoraz} = 30$, $\text{ValorVoraz} = 54$. Capacidad remanente: 70.

2. Detección del Elemento Crítico:

- **Elemento 5:** $w_5 = 75$. Como $75 > 70$ (espacio remanente), **no entra**.
- Se registra $\text{ElementoCritico} = \text{Elemento 5}$ ($v_c = 110$, $w_c = 75$).

3. Continuación del recorrido voraz:

- **Elemento 6:** $w_6 = 40 \leq 70$. Entra. $\text{PesoVoraz} = 70$, $\text{ValorVoraz} = 94$. Capacidad remanente: 30.
- **Elemento 7:** $w_7 = 40 > 30$. No entra.
- Fin del recorrido: Solución voraz acumulada $P_1 = 94$ con peso 70.

4. Evaluación de la Garantía de Calidad:

- Se evalúa: $v_c (110) > \text{ValorVoraz} (94) \rightarrow \text{Verdadero}$.
- **Se activa la garantía de calidad:** se descarta la solución voraz $S_1 = \{1, 2, 3, 4, 6\}$ y se toma únicamente $\{5\}$.

Resultado Obtenido

- **Elementos seleccionados:** [5]
- **Peso utilizado:** 75/100
- **Valor total obtenido:** 110
- **¿Actuó la garantía de calidad?:** SÍ
- **Conclusión del seguimiento:** Sin la garantía de calidad, el voraz simple se habría conformado con $P_1 = 94$. La garantía rescató la solución devolviendo un valor de 110 (que coincide con el óptimo global de este conjunto).

3.4. Complejidad Temporal

El análisis del orden de complejidad temporal se desglosa según cada etapa lógica del algoritmo:

- **Paso 1: Filtrado de elementos no viables:**
Se recorre la colección de n elementos realizando una comparación $w_i \leq W$ por elemento.

$$\text{Costo} = c_1 \cdot n = \mathcal{O}(n)$$

- **Paso 2: Ordenamiento de candidatos:**
Se ordenan a lo sumo n elementos según la clave calculada v_i/w_i . Un algoritmo de ordenamiento basado en comparaciones óptimo (como Timsort o Mergesort) tiene una complejidad en el peor y caso promedio de:

$$\text{Costo} = c_2 \cdot n \log_2(n) = \mathcal{O}(n \log n)$$

- **Paso 3: Inicialización de variables:**
Asignaciones en tiempo constante:

$$\text{Costo} = \mathcal{O}(1)$$

- **Paso 4: Recorrido y llenado voraz:**
El bucle itera exactamente una vez por cada elemento candidato ($m \leq n$). Dentro del bucle se ejecutan comparaciones aritméticas y sumas escalares de costo constante $\mathcal{O}(1)$. Por lo tanto:

$$\text{Costo} = \sum_{i=1}^m \mathcal{O}(1) = \mathcal{O}(m) \leq \mathcal{O}(n)$$

■ **Paso 5: Evaluación de la garantía y selección final:**

Comparación única entre dos enteros $v_c > ValorVoraz$ y construcción del objeto resultado:

$$Costo = \mathcal{O}(1)$$

Cálculo de la Complejidad Temporal Total

Sumando los términos:

$$T(n) = \mathcal{O}(n) + \mathcal{O}(n \log n) + \mathcal{O}(1) + \mathcal{O}(n) + \mathcal{O}(1) = \boxed{\mathcal{O}(n \log n)}$$

El orden de complejidad temporal del algoritmo voraz está **estrictamente acotado por el paso de ordenamiento**, resultando en:

$$\boxed{T(n) = \Theta(n \log n)}$$

Complejidad Espacial:

Se almacena la lista de elementos candidatos y la lista de objetos seleccionados, requiriendo un espacio adicional lineal proporcional a la cantidad de objetos: $\boxed{\mathcal{O}(n)}$.

3.5. Sets de Datos

- **Origen:** Se utilizaron instancias generadas mediante la función provista por la cátedra en `crear_mochila.py`:

- Pesos aleatorios enteros en el intervalo uniforme $[1, 200]$.
- Beneficios aleatorios enteros en el intervalo uniforme $[1, 1000]$.
- Capacidad establecida según la relación de diseño de la cátedra:

$$W = n \times 50$$

Dado que el peso promedio de cada objeto es $(1 + 200)/2 \approx 100$, esta capacidad asegura que aproximadamente el 50 % de los elementos quepan en la mochila, representando un escenario balanceado y no trivial.

- **Criterio de selección de tamaños (N):**

Dado que la complejidad temporal $\mathcal{O}(n \log n)$ es cuasi-lineal y sumamente veloz, evaluar tamaños pequeños (como $N \leq 100$) resultaría en tiempos del orden de microsegundos, indetectables e inundados por el ruido del sistema operativo. Por lo tanto, se seleccionaron tamaños en una escala amplia desde $N = 1,000$ hasta $N = 200,000$:

$$N \in \{1,000, 5,000, 10,000, 25,000, 50,000, 75,000, 100,000, 150,000, 200,000\}$$

- **Protocolo de medición:** Para cada tamaño N se realizaron **5 ejecuciones independientes**. Cada corrida midió estrictamente el tiempo de ejecución de `resolver_mochila_greedy` utilizando el reloj de alta resolución `time.perf_counter()`. Se computó el promedio aritmético de las 5 corridas para aislar variaciones causadas por interrupciones del sistema operativo.

3.6. Tiempos de Ejecución

A continuación se detallan los valores medidos y los valores teóricos ajustados por mínimos cuadrados a la curva $t_{\text{teórico}}(n) = c \cdot n \log_2(n)$:

N	Capacidad W	Tiempo real promedio (ms)	Tiempo teórico ajustado (ms)	Diferencia relativa
1.000	50.000	0.340	0.675	49.62 %
5.000	250.000	2.055	4.161	50.61 %
10.000	500.000	4.675	8.998	48.05 %
25.000	1.250.000	26.003	24.734	5.13 %
50.000	2.500.000	62.370	52.854	18.00 %
75.000	3.750.000	71.970	82.253	12.50 %
100.000	5.000.000	115.843	112.481	2.99 %
150.000	7.500.000	174.031	174.663	0.36 %
200.000	10.000.000	238.889	238.505	0.16 %

Gráfico Comparativo de Tiempos (Curva Medida vs. Curva Teórica de Ajuste)

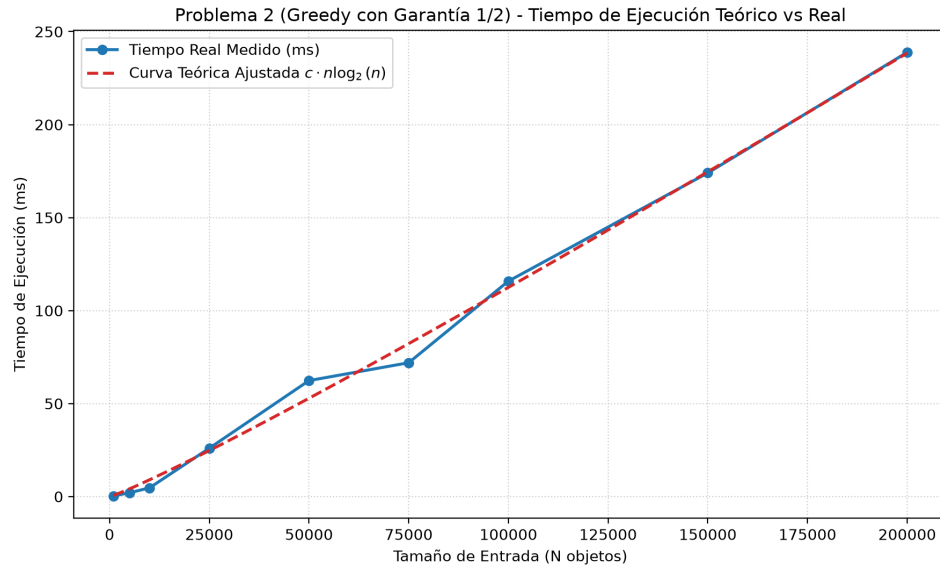


Figura 2: Tiempos de ejecución medidos experimentalmente (puntos azules) contrastados con la curva teórica de ajuste por mínimos cuadrados $c \cdot n \log_2(n)$ (línea roja discontinua).

3.7. Informe de Resultados

¿Se corresponde con la complejidad determinada inicialmente?

Sí, los resultados experimentales confirman de manera contundente la complejidad teórica $\mathcal{O}(n \log n)$ calculada a partir del pseudocódigo.

1. Concordancia asintótica:

Como se observa en la Figura 2, a medida que el tamaño N crece hacia el rango asintótico ($N \geq 75,000$), los puntos experimentales convergen estrechamente sobre la curva teórica de ajuste $c \cdot n \log_2(n)$. Para $N = 100,000$ la diferencia es de apenas el 2.99 %, y para $N \geq 150,000$ desciende a menos del **0.4 %** (0.36 % en 150,000 y 0.16 % en 200,000).

2. Desviaciones en tamaños pequeños ($N \leq 10,000$):

Para valores reducidos de N , el tiempo absoluto de cómputo es menor a 5 milisegundos. En esta franja, el costo fijo del llamado a funciones en Python, la gestión de memoria interna y la recolección de basura representan una fracción visible del tiempo total, produciendo una desviación porcentual mayor pero en términos absolutos despreciable (fracciones de milisegundo).

3. Escalabilidad y desempeño:

La prueba más exigente ($N = 200,000$ elementos y capacidad $W = 10,000,000$)

fue resuelta en apenas **~239 milisegundos**. Esto pone de manifiesto la principal virtud del paradigma Greedy: sacrificar la garantía de optimalidad absoluta (obteniendo a cambio una cota asegurada del 50 %) para lograr tiempos de respuesta órdenes de magnitud más veloces que cualquier método exacto.

4. Programación Dinámica

4.1. Identificación de supuestos, condiciones, limitaciones y/o premisas.

Ambos algoritmos desarrollados resuelven el mismo problema de *Mochila 0/1*, por lo que comparten una serie de supuestos generales. Sin embargo, cada resolución presenta algunas condiciones y limitaciones particulares asociadas a la variable sobre la cual se construyen los sub-problemas.

Se consideran los siguientes supuestos para ambas resoluciones:

- Se cuenta con una cantidad finita de elementos.
- Cada elemento tiene un peso y un beneficio asociados.
- Los pesos y beneficios asociados a cada elementos son valores enteros positivos.
- Cada elemento puede ser seleccionado como máximo una vez, es decir, se resuelve la variante *0/1 (incluir o no incluir)* del problema de la mochila.
- Los elementos no pueden fraccionarse.
- La mochila tiene una capacidad máxima finita y positiva.
- La suma de los pesos de los elementos seleccionados no puede superar la capacidad máxima de la mochila.
- El objetivo global del problema es encontrar una combinación de elementos que maximice el beneficio total obtenido.
- Los datos de entrada se reciben mediante un archivo de texto en el cual la primera línea contiene la capacidad máxima de la mochila y las líneas siguientes contienen pares de la forma:

(peso, beneficio).

4.1.1. Consideraciones del algoritmo tradicional.

El algoritmo tradicional plantea los sub-problemas en función de la capacidad disponible de la mochila.

Se define un estado de la forma:

$$OPT(i, c)$$

que representa el máximo beneficio que puede obtenerse considerando los primeros i elementos y una capacidad disponible c .

4.1.2. Consideraciones del algoritmo alternativo.

El algoritmo alternativo plantea los sub-problemas en función del beneficio que se quiere alcanzar.

Se define un estado de la forma:

$$OPT(i, v)$$

que representa el mínimo peso necesario para alcanzar un beneficio v utilizando los primeros i elementos.

4.2. Diseño.

4.2.1. Pseudocódigo del algoritmo tradicional.

En esta solución tradicional se construye una matriz en la que cada posición representa el máximo beneficio que puede obtenerse para una determinada capacidad de mochila, considerando una cierta cantidad de elementos.

```
1  FUNCION mochila_tradicional(elementos, capacidad):
2      n = cantidad de elementos
3
4      CREAR matriz_optimos de tamaño (n+1) *
        (capacidad+1)
5      RELLENAR todas las posiciones con 0
6
7      PARA i DESDE 1 HASTA n:
8          peso = elementos[i-1].peso
9          beneficio = elementos[i-1].beneficio
10
11         PARA c DESDE 1 HASTA capacidad:
12
13             SI peso > c:
14                 matriz_optimos[i][c] =
                    matriz_optimos[i-1][c]
15
16             SINO:
17                 no_incluir = matriz_optimos[i-1][c]
18                 incluir = beneficio +
                    matriz_optimos[i-1][c-peso]
19
20                 matriz_optimos[i][c] = max(no_incluir,
                    incluir)
21
22         RETORNAR matriz_optimos[n][capacidad]
23  FIN FUNCION
```

4.2.2. Pseudocódigo del algoritmo alternativo.

En cambio, en esta solución alternativa se cambia la perspectiva del problema. En lugar de responder cuánto beneficio máximo puede obtenerse para una capacidad determinada, se obtiene cuál es el peso mínimo necesario para alcanzar un determinado beneficio.

```
1  FUNCION mochila_alternativa(elementos , capacidad):
2      n = cantidad de elementos
3
4      beneficio_total = suma de los beneficios de todos
5                          los elementos
6
7      CREAR matriz_pesos de tamaño (n+1) *
8                          (beneficio_total+1)
9      RELLENAR todas las posiciones con inf
10
11      PARA i DESDE 0 HASTA n:
12          matriz_pesos[i][0] = 0
13
14      PARA i DESDE 1 HASTA n:
15          peso = elementos[i-1].peso
16          beneficio = elementos[i-1].beneficio
17
18          PARA v DESDE 1 HASTA beneficio_total:
19
20              SI beneficio > v:
21                  matriz_pesos[i][v] =
22                      matriz_pesos[i-1][v]
23
24              SINO:
25                  no_incluir = matriz_pesos[i-1][v]
26                  incluir = peso +
27                      matriz_pesos[i-1][v-beneficio]
28
29                  matriz_pesos[i][v] = min(no_incluir ,
30                      incluir)
31
32      PARA v DESDE beneficio_total HASTA 0:
33          SI matriz_pesos[n][v] ≤ capacidad:
34              RETORNAR v
35
36  FIN FUNCION
```

4.2.3. Estructuras de datos utilizadas.

Ambos algoritmos utilizan una matriz bidimensional para almacenar las soluciones óptimas de los distintos sub-problemas. Las filas de la matriz representan la cantidad de elementos considerados, desde 0 hasta n , donde n es la cantidad total de elementos.

La diferencia principal entre ambas implementaciones se encuentra en el significado de las columnas y en el valor almacenado en cada posición.

En el **algoritmo tradicional** se utiliza una matriz de dimensiones:

$$(n + 1) * (C + 1)$$

donde C representa la capacidad máxima de la mochila.

Cada posición de la matriz

$$matriz_optimos[i][c]$$

almacena el máximo beneficio que puede obtenerse considerando los primeros i elementos y disponiendo de una capacidad c .

La matriz se inicializa con ceros, ya que si no se consideran elementos o si la capacidad disponible es cero, el beneficio máximo posible también es cero.

En el **algoritmo alternativo** se utiliza una matriz de dimensiones:

$$(n + 1) * (B + 1)$$

donde

$$B = \sum_{i=1}^n b_i$$

representa la suma de los beneficios de todos los elementos.

Cada posición

$$matriz_pesos[i][v]$$

almacena el mínimo peso necesario para alcanzar exactamente un beneficio v , considerando los primeros i elementos.

La matriz se inicializa con un valor infinito para representar aquellos beneficios que inicialmente no pueden alcanzarse y la primera columna se inicializa en cero, ya que obtener un beneficio igual a cero requiere un peso igual a cero.

4.3. Ejemplos de seguimiento.

Para realizar el seguimiento de los ejemplos se utiliza el siguiente conjunto reducido de datos:

Elemento	Peso	Beneficio
1	2	3
2	3	4
3	4	5

La capacidad máxima de la mochila es:

$$C = 5$$

4.3.1. Ejemplo para el algoritmo tradicional.

El algoritmo tradicional define:

$$OPT(i, c)$$

como el máximo beneficio que puede obtenerse utilizando los primeros i elementos y una mochila con capacidad c .

La matriz posee $(n + 1)$ filas y $(C + 1)$ columnas. En este caso:

$$(3 + 1) \times (5 + 1) = 4 \times 6$$

Inicialmente, la matriz se encuentra llena de ceros:

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0

La fila 0 representa el caso en el cual no se dispone de ningún elemento. Por lo tanto, el beneficio máximo es cero para cualquier capacidad.

Elemento 1

El primer elemento tiene peso 2 y beneficio 3.

Para capacidades menores que 2, el elemento no puede incluirse.

Para una capacidad igual o mayor que 2, el elemento puede incluirse. Por ejemplo:

$$OPT(1, 2) = \max(OPT(0, 2), 3 + OPT(0, 0))$$

$$OPT(1, 2) = \max(0, 3) = 3$$

Para capacidad 3:

$$OPT(1, 3) = \max(OPT(0, 3), 3 + OPT(0, 1)) = 3$$

El mismo razonamiento se aplica para las capacidades 4 y 5.

La primera fila queda:

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3

Elemento 2

El segundo elemento tiene peso 3 y beneficio 4.

Para capacidades menores que 3, el elemento no puede incluirse, por lo que se mantienen los valores de la fila anterior.

Para capacidad 3 se comparan las dos alternativas:

$$OPT(2, 3) = \text{máx}(OPT(1, 3), 4 + OPT(1, 0))$$

$$OPT(2, 3) = \text{máx}(3, 4) = 4$$

Para capacidad 4:

$$OPT(2, 4) = \text{máx}(OPT(1, 4), 4 + OPT(1, 1))$$

$$OPT(2, 4) = \text{máx}(3, 4) = 4$$

Para capacidad 5:

$$OPT(2, 5) = \text{máx}(OPT(1, 5), 4 + OPT(1, 2))$$

$$OPT(2, 5) = \text{máx}(3, 4 + 3) = 7$$

En este caso, conviene incluir el elemento 2 junto con el elemento 1.

La matriz queda:

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7

Elemento 3

El tercer elemento tiene peso 4 y beneficio 5.

Para capacidades menores que 4, se mantienen los valores anteriores.

Para capacidad 4:

$$OPT(3, 4) = \text{máx}(OPT(2, 4), 5 + OPT(2, 0))$$

$$OPT(3, 4) = \text{máx}(4, 5) = 5$$

Para capacidad 5:

$$OPT(3, 5) = \text{máx}(OPT(2, 5), 5 + OPT(2, 1))$$

$$OPT(3, 5) = \text{máx}(7, 5) = 7$$

Por lo tanto, el elemento 3 no mejora la solución previamente obtenida para capacidad 5.

La matriz final es:

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	3	3	3	3
2	0	0	3	4	4	7
3	0	0	3	4	5	7

La solución final se encuentra en:

$$OPT(3, 5) = 7$$

Por lo tanto, el beneficio máximo obtenido es:

$$\boxed{7}$$

y se alcanza seleccionando los elementos 1 y 2, cuyos pesos suman:

$$2 + 3 = 5$$

sin superar la capacidad máxima de la mochila.

4.3.2. Ejemplo para el algoritmo alternativo.

En este planteo se define:

$$OPT(i, v)$$

como el mínimo peso necesario para alcanzar exactamente un beneficio v utilizando los primeros i elementos.

La suma total de beneficios es:

$$B = 3 + 4 + 5 = 12$$

Por lo tanto, la matriz tiene:

$$(3 + 1) \times (12 + 1)$$

posiciones.

En este algoritmo, obtener beneficio cero requiere peso cero:

$$OPT(i, 0) = 0$$

para todo i .

En cambio, cualquier beneficio positivo es inicialmente imposible de alcanzar sin elementos, por lo que:

$$OPT(0, v) = \infty$$

para todo $v > 0$.

La fila inicial queda:

	0	1	2	3	4	5	6	7	8	9	10	11	12
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞

Elemento 1

El primer elemento posee peso 2 y beneficio 3.

Si no se incluye, los beneficios positivos continúan siendo imposibles de alcanzar.

Si se incluye, puede alcanzarse un beneficio igual a 3 con peso 2:

$$OPT(1, 3) = \min(OPT(0, 3), 2 + OPT(0, 0))$$

$$OPT(1, 3) = \min(\infty, 2) = 2$$

Por lo tanto, la fila correspondiente al primer elemento queda:

	0	1	2	3	4	5	6	7	8	9	10	11	12
1	0	∞	∞	2	∞	∞	∞	∞	∞	∞	∞	∞	∞

Elemento 2

El segundo elemento posee peso 3 y beneficio 4.

Al incluirlo de manera individual se obtiene:

$$OPT(2, 4) = \min(OPT(1, 4), 3 + OPT(1, 0))$$

$$OPT(2, 4) = \min(\infty, 3) = 3$$

También puede combinarse con el elemento 1 para alcanzar beneficio 7:

$$OPT(2, 7) = \min(OPT(1, 7), 3 + OPT(1, 3))$$

$$OPT(2, 7) = \min(\infty, 3 + 2) = 5$$

Los beneficios alcanzables luego de considerar los primeros dos elementos son:

0, 3, 4, 7

con pesos mínimos:

$$0, 2, 3, 5$$

respectivamente.

La fila queda:

	0	1	2	3	4	5	6	7	8	9	10	11	12
2	0	∞	∞	2	3	∞	∞	5	∞	∞	∞	∞	∞

Elemento 3

El tercer elemento posee peso 4 y beneficio 5.

Al incluirlo de manera individual:

$$OPT(3, 5) = \min(OPT(2, 5), 4 + OPT(2, 0))$$

$$OPT(3, 5) = 4$$

También puede combinarse con el elemento 1:

$$OPT(3, 8) = \min(OPT(2, 8), 4 + OPT(2, 3))$$

$$OPT(3, 8) = \min(\infty, 4 + 2) = 6$$

Puede combinarse con el elemento 2:

$$OPT(3, 9) = \min(OPT(2, 9), 4 + OPT(2, 4))$$

$$OPT(3, 9) = 7$$

Y combinando los tres elementos:

$$OPT(3, 12) = \min(OPT(2, 12), 4 + OPT(2, 7))$$

$$OPT(3, 12) = 9$$

La fila final queda:

	0	1	2	3	4	5	6	7	8	9	10	11	12
3	0	∞	∞	2	3	4	∞	5	6	7	∞	∞	9

Una vez completada la matriz, se busca el mayor beneficio cuyo peso mínimo no supere la capacidad máxima:

$$OPT(3, v) \leq 5$$

Se observa que:

$$OPT(3, 7) = 5$$

mientras que:

$$OPT(3, 8) = 6 > 5$$

Por lo tanto, el mayor beneficio alcanzable sin superar la capacidad es:

$$\boxed{7}$$

Este beneficio se obtiene seleccionando nuevamente los elementos 1 y 2, cuyos pesos suman:

$$2 + 3 = 5$$

4.4. Complejidad temporal.

A partir de los pseudocódigos desarrollados, se analiza la cantidad de operaciones realizadas por cada algoritmo en función del tamaño de la entrada.

4.4.1. Complejidad del algoritmo tradicional.

Esta resolución utiliza una matriz en la que las filas representan la cantidad de elementos considerados y las columnas representan las capacidades posibles de la mochila.

El pseudocódigo contiene dos ciclos anidados:

```
1 PARA i DESDE 1 HASTA n:  
2     PARA c DESDE 1 HASTA capacidad:  
3         ...
```

El ciclo externo se ejecuta n veces, mientras que el ciclo interno se ejecuta C veces para cada elemento.

Por lo tanto, la cantidad total de iteraciones es:

$$n \cdot C$$

Dentro de cada iteración se realizan únicamente operaciones de tiempo constante, tales como:

- Comparaciones.
- Accesos a posiciones de la matriz.
- Sumas.
- Una operación de máximo.

En consecuencia, la complejidad temporal del algoritmo tradicional es:

$$O(nC)$$

Además, la ecuación de recurrencia utilizada es:

$$DP[i][c] = \begin{cases} DP[i-1][c], & \text{si } p_i > c, \\ \max(DP[i-1][c], b_i + DP[i-1][c - p_i]), & \text{si } p_i \leq c. \end{cases}$$

donde p_i representa el peso del elemento i y b_i su beneficio.

Los casos base son:

$$DP[0][c] = 0$$

para toda capacidad c , y

$$DP[i][0] = 0$$

para todo i .

4.4.2. Complejidad del algoritmo alternativo.

En esta resolución se modifica la dimensión sobre la cual se construyen los subproblemas. En lugar de utilizar las capacidades posibles de la mochila como columnas, se utilizan los posibles valores de beneficio.

El pseudocódigo contiene nuevamente dos ciclos anidados:

```
1 PARA i DESDE 1 HASTA n:  
2     PARA v DESDE 1 HASTA beneficio_total:  
3         ...
```

El ciclo externo se ejecuta n veces y el interno se ejecuta B veces para cada elemento.

Por lo tanto, la cantidad principal de operaciones es:

$$n \cdot B$$

Dentro de cada iteración se realizan operaciones de tiempo constante, tales como:

- Comparaciones.
- Accesos a la matriz.
- Sumas.
- Una operación de mínimo.

Por lo tanto, la complejidad temporal correspondiente al llenado de la matriz es:

$$\boxed{O(nB)}$$

Una vez construida la matriz, el algoritmo realiza un recorrido adicional desde el beneficio total B hasta 0 para encontrar el mayor beneficio cuyo peso mínimo no supere la capacidad de la mochila.

Este recorrido tiene complejidad:

$$O(B)$$

Por lo tanto, la complejidad temporal total puede expresarse como:

$$O(nB + B)$$

Como el término dominante es nB , para $n > 1$ se obtiene:

$$\boxed{O(nB)}$$

Además, la ecuación de recurrencia utilizada es:

$$DP[i][v] = \begin{cases} DP[i-1][v], & \text{si } b_i > v, \\ \min(DP[i-1][v], p_i + DP[i-1][v - b_i]), & \text{si } b_i \leq v. \end{cases}$$

Los casos base son:

$$DP[0][0] = 0,$$

$$DP[0][v] = \infty \quad \text{para } v > 0.$$

Una vez completada la matriz, se busca el mayor beneficio v tal que:

$$DP[n][v] \leq C.$$

4.5. Tiempos de ejecución.

4.5.1. Set de Datos.

Los conjuntos de datos utilizados fueron generados con el código provisto por la cátedra `crear_mochila.py` y se ejecutaron exactamente los mismos datos para ambos algoritmos, con el objetivo de que la comparación de tiempos sea directa y no

esté condicionada por diferencias en las entradas. Se eligieron instancias de tamaños crecientes para observar cómo evoluciona el tiempo de ejecución al aumentar la cantidad de elementos.

Los tamaños seleccionados se limitaron a valores que pudieran ser procesados por ambas resoluciones, ya que la formulación alternativa requiere una cantidad de memoria significativamente mayor para instancias grandes del problema.

Los conjuntos de datos utilizados se resumen debajo:

N	Capacidad C	Beneficio total B	Archivo utilizado
10	500	5237	mochila10.txt
20	1000	9107	mochila20.txt
50	2500	25657	mochila50.txt
100	5000	52044	mochila100.txt
200	10000	94222	mochila200.txt
300	15000	147832	mochila300.txt
400	20000	200379	mochila400.txt
500	25000	259157	mochila500.txt
600	30000	290282	mochila600.txt
800	40000	399074	mochila800.txt
1000	50000	504570	mochila1000.txt

Cuadro 1: Conjuntos de datos utilizados para la evaluación de ambos algoritmos.

4.5.2. Resultados.

Para el **algoritmo tradicional** las mediciones de tiempos de ejecución arrojaron los siguientes resultados:

N	Capacidad C	Beneficio total B	Tiempo promedio (s)
10	500	5237	0.000846
20	1000	9107	0.002820
50	2500	25657	0.016480
100	5000	52044	0.069402
200	10000	94222	0.285869
300	15000	147832	0.645982
400	20000	200379	1.172393
500	25000	259157	1.967031
600	30000	290282	2.956460
800	40000	399074	5.238829
1000	50000	504570	8.624643

Cuadro 2: Tiempos de ejecución medidos para el algoritmo tradicional.

Para el **algoritmo alternativo** las mediciones de tiempos de ejecución arrojaron los siguientes resultados:

N	Capacidad C	Beneficio total B	Tiempo promedio (s)
10	500	5237	0.007841
20	1000	9107	0.025833
50	2500	25657	0.178003
100	5000	52044	0.707348
200	10000	94222	2.570107
300	15000	147832	6.106540
400	20000	200379	11.526886
500	25000	259157	18.988357
600	30000	290282	25.454690
800	40000	399074	51.501221
1000	50000	504570	93.315060

Cuadro 3: Tiempos de ejecución medidos para el algoritmo alternativo.

4.5.3. Gráficos.

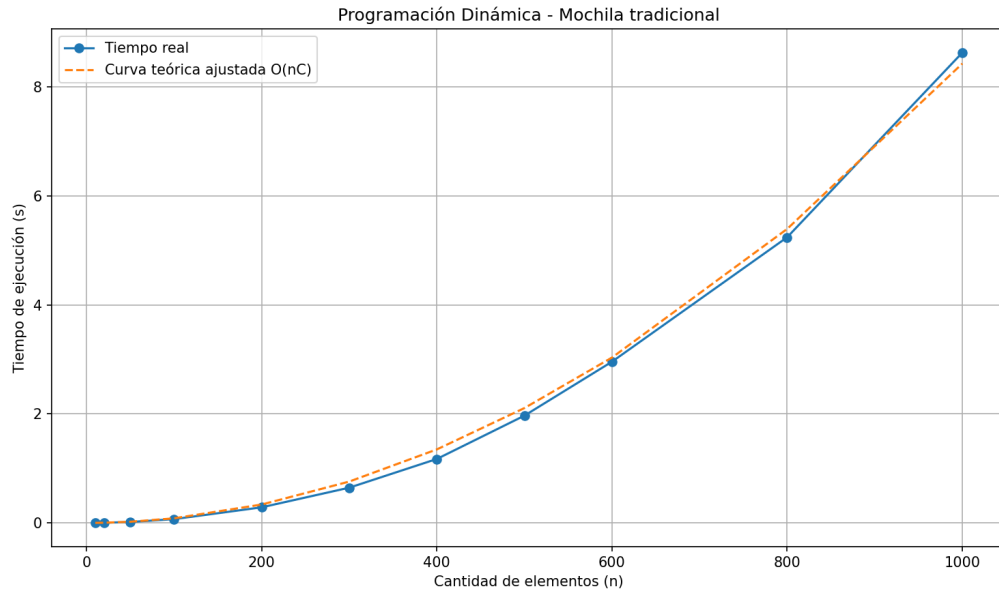


Figura 3: Comparación entre el tiempo de ejecución medido y la curva teórica ajustada para el algoritmo tradicional.

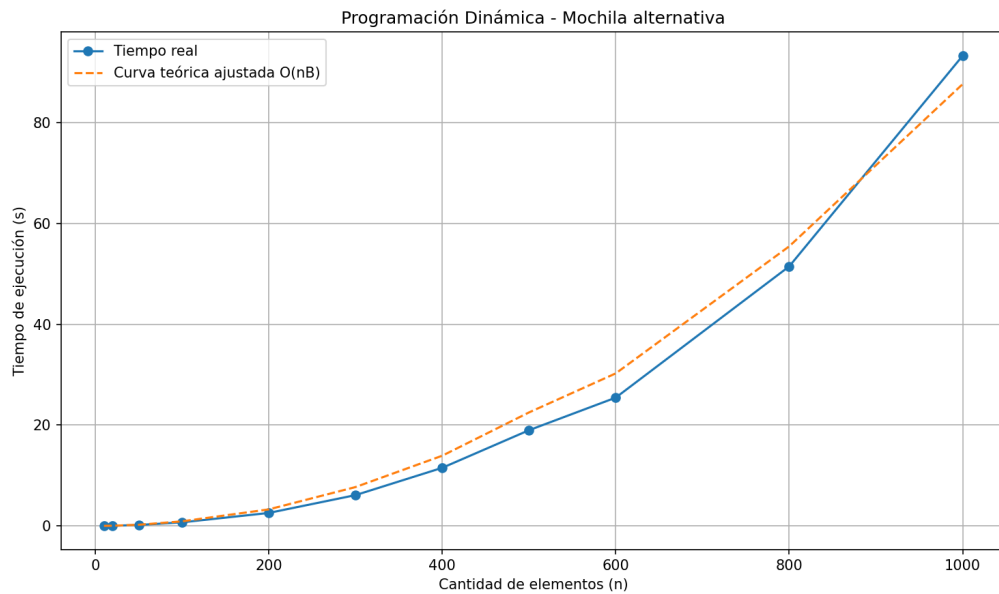


Figura 4: Comparación entre el tiempo de ejecución medido y la curva teórica ajustada para el algoritmo alternativo.

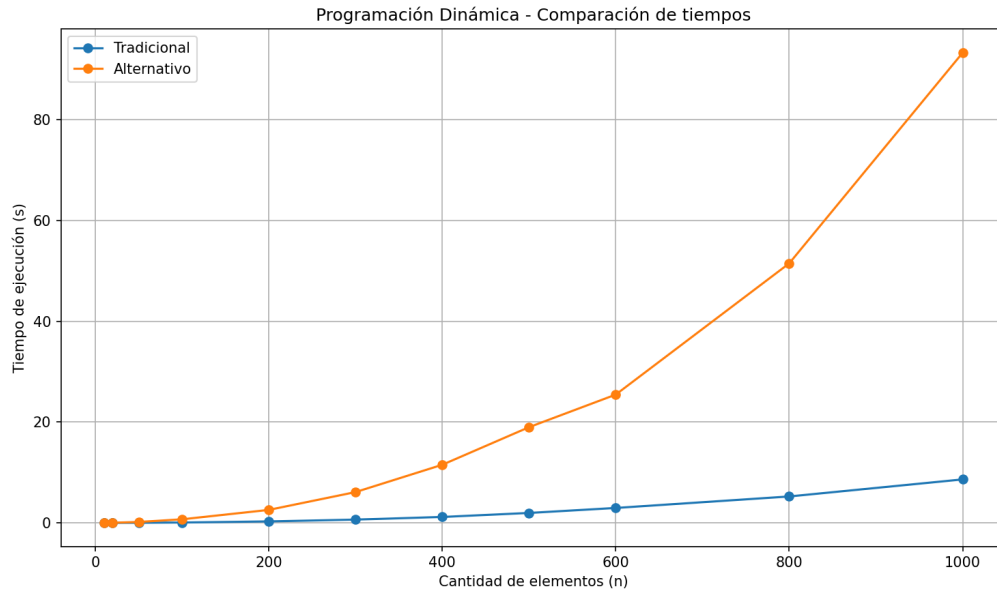


Figura 5: Comparación de los tiempos de ejecución de ambos algoritmos de Programación Dinámica.

4.6. Informe de resultados comparativos: tiempos medidos vs tiempos teóricos.

Los resultados obtenidos en las mediciones muestran que ambas implementaciones presentan un crecimiento consistente con la complejidad temporal determinada teóricamente.

Para el **algoritmo tradicional**, la complejidad obtenida fue:

$$O(nC)$$

donde n representa la cantidad de elementos y C la capacidad de la mochila. En los conjuntos de datos utilizados, la capacidad se define como $C = 50n$, por lo que el costo temporal esperado respecto de n puede expresarse como:

$$O(nC) = O(n \cdot 50n) = O(n^2).$$

Los tiempos medidos presentan efectivamente un crecimiento aproximadamente cuadrático. Por ejemplo, al aumentar el tamaño de la instancia desde $n = 500$ hasta $n = 1000$,

el tiempo promedio pasa de aproximadamente 1,97 segundos a 8,62 segundos, es decir, aumenta alrededor de cuatro veces, comportamiento compatible con una función cuadrática. Los valores medidos siguen además de manera cercana la curva teórica ajustada mostrada en el gráfico correspondiente.

Para el **algoritmo alternativo**, la complejidad obtenida fue:

$$O(nB)$$

donde B es la suma total de los beneficios de los elementos. Debido a que los beneficios individuales se generan dentro de un rango fijo, el valor de B crece aproximadamente de manera lineal con n . Por lo tanto, para los conjuntos de datos utilizados también se espera un comportamiento aproximadamente cuadrático respecto de n .

Los resultados experimentales confirman este comportamiento. El tiempo promedio del algoritmo alternativo pasa de aproximadamente 18,99 segundos para $n = 500$ a 93,32 segundos para $n = 1000$, mostrando un crecimiento compatible con la complejidad $O(nB)$. La curva experimental mantiene además una forma similar a la curva teórica ajustada.

Por último, al comparar ambas implementaciones sobre los mismos conjuntos de datos, se observa que el **algoritmo alternativo** presenta tiempos de ejecución mayores. Para $n = 1000$, por ejemplo, el **algoritmo tradicional** demora aproximadamente 8,62 segundos, mientras que el alternativo demora aproximadamente 93,32 segundos. Esta diferencia se explica por la dimensión utilizada en cada uno. En el tradicional se recorren los estados asociados a las capacidades hasta C , mientras que en el alternativo se recorren los estados asociados a los beneficios hasta B . En las instancias generadas, el valor de B resulta considerablemente mayor que C .

Para $n = 1000$, por ejemplo,

$$C = 50000,$$

mientras que

$$B = 504570.$$

Por este motivo, aunque ambas resoluciones presentan un crecimiento aproximadamente cuadrático respecto de n para estos conjuntos de datos, el algoritmo alternativo procesa una cantidad mucho mayor de estados y, consecuentemente, requiere mayores tiempos de ejecución.

En conclusión, los resultados experimentales son consistentes con las complejidades determinadas inicialmente. Las curvas medidas reproducen la forma de crecimiento esperada para ambos algoritmos, mientras que la diferencia entre sus tiempos se debe principalmente a que, en los datos utilizados, la suma total de beneficios B es significativamente mayor que la capacidad C .

5. Programación Lineal Entera

5.1. Supuestos

Para el diseño, implementación y análisis del algoritmo de programación lineal entera, se establecen los siguientes supuestos, condiciones y limitaciones:

- **Indivisibilidad de objetos (0/1):** cada elemento debe incluirse o descartarse. No se permite el fraccionamiento, convirtiendo el problema en un Programa Lineal Entero Binario.
- **Positividad de parámetros:** la capacidad W , los pesos w_i y los beneficios v_i son enteros estrictamente positivos.
- **Solver utilizado:** se emplea la biblioteca PuLP con el solver, el cual es COIN-OR Branch and Cut (CBC, incluido por defecto).
- **Dependencia del solver externo:** el tiempo de ejecución abarca la resolución interna, no incluye el tiempo de armado del modelo en PuLP.

5.2. Diseño

5.2.1. Modelado Matemático

Variables de decisión:

$$x_i \in \{0, 1\}, \quad i = 1, \dots, n$$

donde $x_i = 1$ indica que el elemento i es incluido en la mochila, y $x_i = 0$ que no lo es.

Función objetivo:

$$\text{Maximizar } Z = \sum_{i=1}^n v_i \cdot x_i$$

Restricción de capacidad: se quiere poder agregar todos los elementos posibles en la mochila tal que se obtenga la máxima ganancia, sin superar la capacidad.

$$\sum_{i=1}^n w_i \cdot x_i \leq W$$

Restricción de integralidad: esto permite definir las variables como variables enteras binarias.

$$x_i \in \{0, 1\}, \quad \forall i = 1, \dots, n$$

5.2.2. Pseudocódigo

```
1  Algoritmo MochilaProgramacionLineal(valores, pesos, W):
2      n = longitud(valores)
3      modelo = NuevoProblema(tipo=MAXIMIZAR)
4
5      Para i desde 0 hasta n-1:
6          X[i] = NuevaVariableBinaria("X_i")
7
8      AgregarObjetivo(modelo, sumatoria(valores[i] * X[i]
9          para i en 0..n-1))
10     AgregarRestriccion(modelo, sumatoria(pesos[i] *
11         X[i] para i en 0..n-1) ≤ W)
12
13     Resolver(modelo, solver=CBC)
14
15     seleccionados = []
16     Para i desde 0 hasta n-1:
17         Si valor(X[i]) = 1:
18             seleccionados.Agregar(i)
19
20     Retornar seleccionados
```

5.3. Seguimiento con Set Reducido

Se tiene el siguiente archivo `mochila10.txt`, que muestra inicialmente la capacidad de la mochila y luego los pares `peso,valor`, uno por línea:

```
500
69,56
186,525
124,876
95,417
9,942
36,163
106,851
47,989
89,451
162,2
```

Se utiliza el conjunto de 10 elementos mencionado anteriormente con capacidad

$W = 500$:

Elemento i	Peso w_i	Valor v_i	Densidad v_i/w_i
1	69	56	0.81
2	186	525	2.82
3	124	876	7.06
4	95	417	4.39
5	9	942	104.67
6	36	163	4.53
7	106	851	8.03
8	47	989	21.04
9	89	451	5.07
10	162	2	0.01

Formulación del problema:

$$Z = 56X_1 + 525X_2 + 876X_3 + 417X_4 + 942X_5 + 163X_6 + 851X_7 + 989X_8 + 451X_9 + 2X_{10}$$

Resolución por solver: los elementos excluidos son el 1 (peso 69, valor 56), el 2 (peso 186, valor 525), el 6 (peso 36, valor 163) y el 10 (peso 162, valor 2). La solución elegida pesa 470, por lo que quedan 30 unidades libres en la mochila y ninguno de los elementos excluidos entra, ya que todos pesan más de 30. Cabe destacar que el elemento 6 tiene una densidad (4.53) levemente mayor que la del elemento 4 (4.39), pero el solver prefiere incluir el 4: reemplazarlo por los elementos 6 y 1 daría un peso de 480 y un valor de 4328, inferior a 4526. Esto muestra que ordenar por densidad no garantiza el óptimo.

Elemento i	Peso w_i	Valor v_i	Seleccionado
1	69	56	No
2	186	525	No
3	124	876	Sí
4	95	417	Sí
5	9	942	Sí
6	36	163	No
7	106	851	Sí
8	47	989	Sí
9	89	451	Sí
10	162	2	No
Total	470	4526	

¿Cómo queda la función objetivo después de ejecutar el programa?

$$\begin{aligned} Z &= 56(0) + 525(0) + 876(1) + 417(1) + 942(1) + 163(0) \\ &\quad + 851(1) + 989(1) + 451(1) + 2(0) = 4526 \end{aligned}$$

Resultado final: valor total = 4526, peso total = 470/500. La solución es óptima y factible.

5.4. Complejidad Temporal

En teoría, el problema de la mochila formulado como Programa Lineal Entero Binario tiene complejidad exponencial. Cada variable X_i puede tomar valor 0 o 1, por lo que para n variables el espacio de soluciones tiene 2^n combinaciones posibles. En el peor caso, el solver debería explorar cada una de ellas para garantizar la optimalidad.

Sin embargo, en la práctica el solver no necesita explorar ese árbol completo, gracias a tres mecanismos:

- **Se establece una cota superior:** antes de aplicar Branch & Cut, el solver resuelve la relajación continua del problema (permitiendo $X_i \in [0, 1]$) mediante el método Simplex. Esta solución continua da una cota superior del valor óptimo entero. Muchas ramas del árbol quedan descartadas inmediatamente si su cota superior no puede superar la mejor solución entera encontrada hasta el momento.
- **Poda agresiva:** cuando en un nodo del árbol la solución LP relajada ya es entera (todas las variables toman valor 0 o 1 naturalmente), no hace falta seguir ramificando.
- **Planos de corte:** el solver agrega restricciones válidas que achican el espacio de soluciones, logrando obtener una solución óptima.

Complejidad total:

$$T(n) = \boxed{\mathcal{O}(2^n)}$$

5.5. Tiempos de Ejecución

A continuación se muestran los resultados con diferentes tamaños de n :

Resultados - Problema 4: Programacion Lineal		
Tiempo medido: solo resolucion del solver		
Repeticiones por instancia: 3		
n	Capacidad W	Tiempo (s)
10	500	0.012559
100	5000	0.015803
1000	50000	0.067902
10000	500000	0.269558
50000	2500000	2.719557
100000	5000000	8.899557
150000	7500000	20.047171

Figura 6: Resultados

Se podría crear un conjunto de datasets con una mayor cantidad de elementos, pero el programa tarda demasiado a partir de $n = 200000$.

Gráfico Comparativo de Tiempos (Curva Medida vs. Curva Teórica de Ajuste)

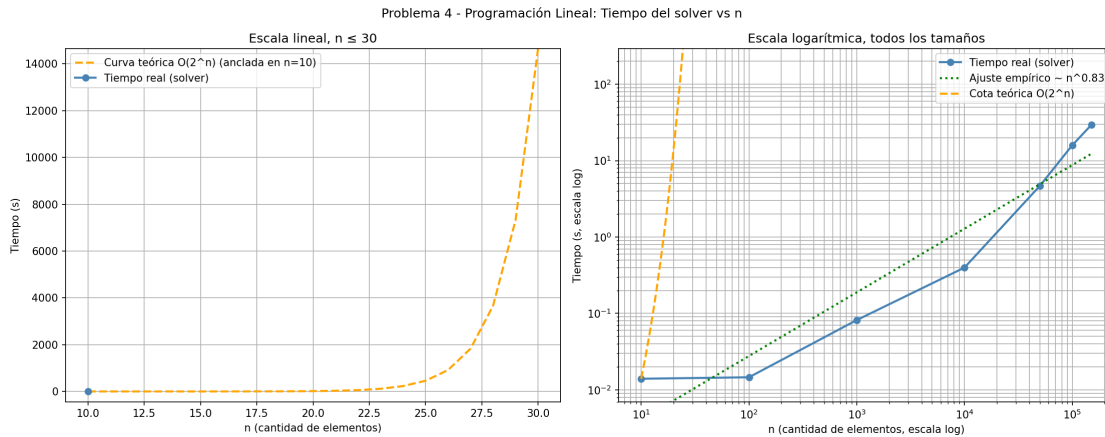


Figura 7: Tiempo del solver en función de n . A la izquierda, la curva teórica $O(2^n)$ (naranja, anclada en el primer punto medido) y los tiempos medidos (azul), en escala lineal y para $n \leq 30$. A la derecha, los tiempos medidos para todos los tamaños, junto con su ajuste empírico $t \approx a \cdot n^b$ (verde) y la cota teórica (naranja), en escala logarítmica en ambos ejes.

La curva teórica $O(2^n)$ crece tan rápido que solo puede apreciarse para valores chicos de n . Por eso, el panel izquierdo la muestra en escala lineal hasta $n = 30$: ahí se ve su forma exponencial, que sube cada vez más rápido, mientras que el tiempo medido del solver queda prácticamente en cero.

Los tamaños evaluados, en cambio, van de $n = 10$ a $n = 150000$, y los tiempos medidos, de milisegundos a segundos. En escala lineal, los puntos de n pequeño quedarían todos pegados al origen y no se distinguirían. Por eso el panel derecho usa escala logarítmica en ambos ejes, lo que permite ver todos los tamaños a la vez.

Si todo se graficara en un único panel con escala logarítmica en el eje vertical, la curva exponencial se vería como una línea recta, porque el logaritmo de 2^n es $n \log 2$, que es una función lineal de n . Esa recta podría confundirse con un crecimiento lineal, cuando en realidad representa un crecimiento exponencial.

A continuación, se deja un ejemplo de cómo se vería:

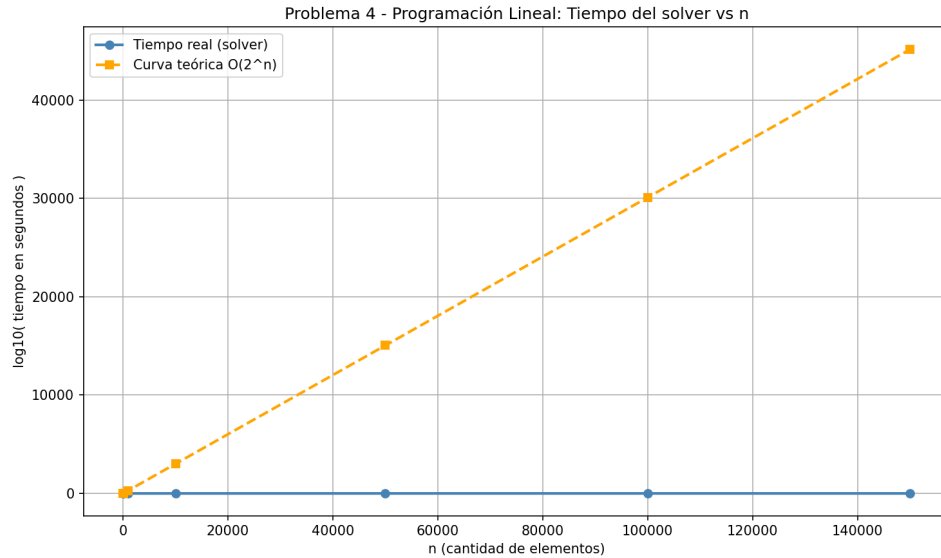


Figura 8: Tiempos de ejecución medidos experimentalmente (puntos azules) contrastados con la curva teórica de ajuste por mínimos cuadrados $O(2^n)$ (línea amarilla discontinua).

5.6. Informe de Resultados

En cuanto a la complejidad, la teoría establece un peor caso de $O(2^n)$ debido al Branch & Cut que el solver debe aplicar. Sin embargo, los tiempos medidos en la práctica revelan un crecimiento significativamente más moderado, gracias a las técnicas de poda y planos de corte que incorpora el solver.

No obstante, esta ventaja práctica tiene un límite. A medida que n crece, el tiempo de resolución comienza a crecer de forma más pronunciada, y para ciertas instancias el comportamiento exponencial teórico puede manifestarse. Por este motivo, la programación lineal entera no resulta adecuada para instancias de escala masiva como las que el algoritmo Greedy resuelve ($n = 200000$ o más), pero sí representa la herramienta poderosa, disponible cuando se requiere una solución óptima en casos de tamaño moderado.

6. Conclusión General

Los resultados muestran que no existe un algoritmo que sea mejor en todos los aspectos ya que cada paradigma resuelve un aspecto distinto como exactitud, tiempo, y memoria.

La poda del backtracking mejora mucho el comportamiento práctico, aunque el tiempo se vuelve dependiente de los datos. La programación dinámica elimina ese crecimiento exponencial a cambio de depender del valor numérico de la capacidad o de los beneficios, y de una memoria que crece con el tamaño de la tabla. La programación lineal entera ofrece soluciones exactas con un esfuerzo de implementación mínimo y un comportamiento práctico muy favorable, aunque para valores de n muy grandes (a partir de $n = 200000$), se nota su crecimiento exponencial. Finalmente, el algoritmo greedy resuelve en poco tiempo instancias que ningún método exacto de los evaluados procesa en un tiempo comparable, y en las instancias analizadas la solución obtenida queda muy cerca del óptimo.

La elección del algoritmo depende del tamaño de la instancia, la capacidad y los beneficios, y de si se requiere de una solución óptima o si admite una aproximación de calidad conocida.

6.1. Comparativa de Complejidades Teóricas

Algoritmo	Paradigma	Complejidad Temporal	Tipo de Solución
Fuerza Bruta	Búsqueda exhaustiva	$\mathcal{O}(2^n)$	Exacta
Backtracking	Búsqueda con poda	$\mathcal{O}(2^n)$ peor caso	Exacta
Greedy con Garantía 1/2	Heurística voraz	$\mathcal{O}(n \log n)$	Aproximación ($\geq 50\%$)
PD - Planteo Tradicional	Programación dinámica	$\mathcal{O}(n \cdot W)$	Exacta (pseudopolinomial)
PD - Planteo Alternativo	Programación dinámica	$\mathcal{O}(n \cdot V)$	Exacta (pseudopolinomial)
Programación Lineal (PuLP)	Branch and Cut	Exponencial peor caso	Exacta